

Finance Research Agent — Updated Phase-Wise Implementation Plan

Execution roadmap for React + FastAPI + Flue + PostgreSQL architecture

Phase 0 — Framework Discovery & Architecture Baseline

- Read both planning PDFs and inspect the actual Flue framework.
- Identify agent, tools, ReAct/workflow, state/context, structured output, errors, logging/tracing, and hook APIs.
- Also inspect constraints for integrating FastAPI, PostgreSQL, and the future React frontend.
- Run a minimal Flue agent/tool example.
- Create docs/flue_framework_notes.md and project baseline documentation.
- Exit: actual framework APIs and hook capabilities are understood; no finance functionality is built.

Phase 1 — Project Foundation + Full Application Skeleton

- Create backend/, frontend/, tests/, docs/, examples/ structure while preserving assets/.
- Initialize FastAPI backend, Pydantic settings, SQLAlchemy, PostgreSQL connection/configuration, and health endpoint.
- Initialize React + Vite + TypeScript frontend and basic API client.
- Configure Docker Compose for backend + PostgreSQL and local development.
- Integrate the actual Flue dependency discovered in Phase 0.
- Exit: frontend, backend, database connection, and minimal Flue example run successfully.

Phase 2 — Market Data Layer + Persistence

- Implement MarketDataProvider with live and mock providers.
- Implement fetch_market_data(symbol).
- Normalize current price, previous close, daily change, 52-week range, volume, recent history, and source.
- Persist stock metadata and market snapshots to PostgreSQL.
- Expose backend plumbing needed by the agent; do not move finance calculations into the API layer.
- Test valid/invalid symbols, missing history, provider failure, and database persistence.

Phase 3 — Deterministic Finance / Indicator Engine

- Implement daily return, 52-week distances, SMA20, SMA50, momentum, and volatility.
- Create typed indicator models.
- Keep calculations independent of the LLM, frontend, and database.
- Persist analysis indicators when an analysis run is executed.
- Test every calculation with deterministic fixtures.

Phase 4 — Bullish + Risk Signal Engine

- Implement analyze_bullish_factors() and analyze_risk_factors().
- Each signal contains name, evidence, and strength.
- Persist structured signals against analysis runs.
- Expose structured signal results to the Flue agent and API.
- Test bullish, bearish, mixed, and incomplete-data scenarios.

Phase 5 — Recommendation + Enhanced Confidence

- Implement signal score and recommendation thresholds.
- Implement BUY/HOLD/WATCHLIST/AVOID.
- Implement confidence weighting: 30/25/20/15/10.
- Validate 0–100 range and missing-data behavior.
- Persist recommendation and confidence.
- Document confidence as evidence strength, not price-rise probability.

Phase 6 — Flue Agent Workflow + Backend API

- Implement the actual Flue agent/tool workflow using Phase 0 findings.
- Wire `fetch_market_data` → `calculate_indicators` → `bullish` → `risk` → `thesis` → `confidence` → `report`.
- Implement FastAPI POST `/api/research`.
- Return structured report JSON to the frontend.
- Persist the completed analysis run and report.
- Test agent workflow and API integration.

Phase 7 — Agent Hooks + Validation

- Implement only hooks supported by actual Flue.
- Add input validation, tool lifecycle observability, reasoning/evidence consistency, confidence attachment, and final report validation where supported.
- Persist useful execution metadata without leaking secrets.
- Test hook behavior and unsupported-hook limitations.

Phase 8 — Investment Thesis + Final Report UI

- Implement structured final report generation.
- Include stock, price, daily change, 52-week range, volume, bullish factors, risk factors, technical/financial view, recommendation, confidence, reasoning summary, and disclaimer.
- Build React research-report UI.
- Display indicators, signals, confidence, and recommendation without duplicating backend calculations.
- Test report schema and frontend rendering/error states.

Phase 9 — Stock Comparison Mode

- Implement POST `/api/compare`.
- Run the same deterministic pipeline independently for both stocks.
- Compare all required metrics, signals, score, and confidence.
- Generate comparison reasoning/report.
- Persist comparison analysis.
- Build React comparison UI.
- Test valid pair, invalid pair, duplicate pair, and missing data.

Phase 10 — Full Testing + Error Handling

- Complete unit, tool, agent/workflow, API, database integration, and end-to-end tests.
- Test provider failures, database failures, malformed agent output, invalid input, unsupported symbols, incomplete history, and frontend API failures.
- Verify deterministic calculations and stable confidence.
- Verify error responses are controlled and user-readable.

Phase 11 — Samples + Documentation + Developer Experience

- Create at least three stock reports and one comparison report.
- Complete README with frontend, backend, PostgreSQL, Flue, hooks, architecture, APIs, setup, Docker, configuration, tests, live/mock data, design decisions, limitations, and disclaimer.
- Document database schema and API contracts.
- Document sample usage for single stock and comparison.
- Clean unused code and dependencies.

Phase 12 — Final Audit & Submission

- Audit every original assignment requirement and every expanded architecture requirement.
- Verify Flue usage, clean tool/agent separation, market data, finance logic, confidence, hooks where supported, report, comparison, frontend, backend, PostgreSQL, tests, documentation, samples, Git branch, and no trading functionality.
- Run the complete test suite and a clean Docker Compose startup.
- Verify working tree, branch, commit history, README, sample reports, and final deliverables.
- Prepare final submission summary.

Master Plan → Phase Plan Mapping

Phase	Primary scope
0	Flue discovery + React/FastAPI/PostgreSQL integration constraints
1	Full skeleton: backend + frontend + PostgreSQL + Flue + Docker
2	Market data + database snapshots
3	Deterministic indicators
4	Bullish/risk signals
5	Recommendation + confidence
6	Flue workflow + research API
7	Flue hooks + validation
8	Final report + React report UI
9	Comparison API + comparison UI
10	Complete test/error layers
11	Reports + docs + API/database/frontend documentation
12	Full architecture and assignment audit